Just before competition time, the phone starts ringing. The download, which had been working fine up to this point, was now taking so long that requests were timing out. The pressure was intense, because the administrators had to use this information to build schedules and send out mailings.

A little experimentation showed that the problem was in the routine that took the results of the database query and generated the CSV download. The code looked something like this:

ruby_style/csv_from_row.rb

```
def csv_from_row(accumulator, row)
  result = ""
  until row.empty?
    entry = row.shift.to_s
    if /[,"]/.match?(entry)
      entry = entry.gsub(/"/, '""')
      result << '"' << entry << '"'
    else
      result << entry
    end
    result << "," unless row.empty?
  end
  accumulator << result << CRLF
end

result = ""
query.each_row { |row| csv_from_row(result, row) }

http.write(result)
```

When this code ran against moderate-size data sets, it performed fine. But at a certain input size, it suddenly slowed right down. The culprit? Garbage collection. The approach was generating thousands of intermediate strings and building one big result string, one line at a time. As the big string grew, it needed more space, and garbage collection was invoked, which necessitated scanning and removing all the intermediate strings.

The answer was simple and surprisingly effective. Rather than build the result string as it went along, the code was changed to store each CSV row as an element in an array. This meant that the intermediate lines were still referenced and hence were no longer garbage. It also meant that we were no longer building an ever-growing string that forced garbage collection. Thanks to duck typing, the change was trivial:

```
def csv_from_row(accumulator, row)
  # as before
end

result = []
query.each_row { |row| csv_from_row(result, row) }

http.write(result.join)
```

All that changed is that we passed an array into the `csv_from_row` method. Because it (implicitly) used duck typing, and the only message passed to the `accumulator` was `<<`, the `csv_from_row` method itself was not modified; it continued to append the data it generated to its accumulator parameter, not caring what type that parameter was. After the method returned its result, we joined all those individual lines into one big string. This one change reduced the time to run from more than three minutes to a few seconds.